

```

243 270 int main(){
244 271     ClearConsoleToColors(15, 1);
245 272     SetConsoleTitle("Project - Programming Code Review Screen");
246 273     int choice;
247 274     char ch = 'a';
248 275     while(1){
249 276         system("cls");
250 277         printf("1. Find Out the Day\n");
251 278         printf("2. Print all the day of month\n");
252 279         printf("3. Add Note\n");
253 280         printf("4. EXIT\n");
254 281         printf("ENTER YOUR CHOICE : ");
255 282         scanf("%d",&choice);
256 283         system("cls");
257         switch(choice){

```

C - exercise book

Piotr Jastrzębski

2026-05-17

Table of contents

C - exercise book	4
I Fundamentals of the C Language	5
1 Input and Output Operations	6
2 Arithmetic Operators	7
3 Conditional statements, conditional operator	8
4 Bitwise operators	10
5 Debugging - basics	11
6 Loops	15
7 Functions	17
8 Pointers	19
9 Function Pointers	22
10 One-Dimensional Arrays	24
11 Strings	32
12 Multidimensional arrays	35
13 Complex data types	42
14 Singly-linked lists	46
14.1 Without functions	46
14.2 Displaying	46
14.3 Creating a list	47
14.4 Adding at the beginning	48
14.5 Adding at the end	48
14.6 Searching/counters	49

14.7 Removing individual elements	51
14.8 Removing the entire list	52
14.9 Miscellaneous exercises	53
Bibliography and Other Problem Sets	57

C - exercise book

Here you will find a collection of programming exercises in the C language. The inspiration was to compile tasks created during teaching classes carried out at the Faculty of Mathematics and Computer Science of the University of Warmia and Mazury in Olsztyn.

Some of the exercises have been machine-translated into English. In case of any ambiguities, please get in touch via email.

Part I

Fundamentals of the C Language

1 Input and Output Operations

1. Write a programme that asks the user to enter a single integer, then displays it on the screen.
2. Create a programme that reads two floating-point numbers from the user and prints their difference.
3. Design an application that asks the user for their year of birth, then prints the previous year (one less).
4. Write a programme that reads three integers from the user and prints their average as a floating-point value.
5. Create a programme that asks the user to enter two letters (characters), then prints them in reverse order.
6. Write a programme that asks the user to enter a single floating-point number, then doubles its value and prints the result.
7. Create a programme that reads a floating-point number from the user representing an amount in dollars, then prints how much that is in euros, using a fixed exchange rate (e.g. 1 dollar = 0.85 euros).
8. Write a programme that displays the following text on the screen: **This is a quote: "I often use the C language."**. Ensure that the quotation marks are correctly displayed as part of the string.
9. Create a programme that prints a Windows folder path to standard output, e.g. `C:\Program Files\MyApplication\` (hardcoded, without retrieving anything from the system) or `C:\\Program Files\\MyApplication\\`.
10. Design a programme that demonstrates how to print the following text on the screen: **Special characters: \t (tab), \n (newline), % (percent), \\ (backslash).**
11. Write a programme that reads two rational numbers from standard input representing the lengths of the legs of a right-angled triangle. Then calculate and display the length of the hypotenuse.
12. Write a programme that reads an integer from standard input and prints its absolute value to standard output.
13. Write a programme that reads a floating-point number from standard input and prints its absolute value to standard output.
14. Find an example and display on standard output a case where the addition of floating-point numbers is not associative.

2 Arithmetic Operators

1. Write a programme that calculates the remainder of dividing the sum of two integers by a third integer.
2. Create a programme that calculates the difference of squares of two given integers.
3. Write a programme that calculates the product of the difference of two integers and a third integer.
4. Design a programme that calculates the geometric mean of the absolute values of three given integers.
5. Write a programme that calculates the square of the sum of two given integers.
6. Create a programme that calculates the sum of squares of three given integers.
7. Write a programme that calculates how many times one given integer fits into another given integer.
8. Design a programme that calculates the square of the difference of two given integers.
9. Write a programme that calculates the quotient of the sum of two integers divided by their difference.
10. Create a programme that calculates the sum of three consecutive integers, starting from a given integer.
11. Find the arithmetic mean of three floating-point numbers.
12. Calculate the value of the expression $\frac{1}{\frac{1}{a} + \frac{1}{b} + \frac{1}{c}}$, where a, b, c are non-zero.
13. Calculate the value of e^x for small values of x , e.g. $|x| < 0.01$, approximating e^x as $1 + x$, without using exponential functions.
14. Calculate the area of a triangle with sides a, b and the angle C between them in degrees, using the formula $0.5 \cdot a \cdot b \cdot \sin C$, assuming $\sin C \approx C$ for small angles in radians.
15. Calculate $\sqrt[3]{x}$ for small values of x , e.g. $|x| < 0.01$, approximating $\sqrt[3]{x} \approx 1 + \frac{1}{3}x$, without using any root functions.

3 Conditional statements, conditional operator

In the following exercises, use `if`, `if else`, or `switch case` as the conditional statement. The conditional operator refers to `?`.

1. Write a programme that asks the user to enter an integer. The programme should display whether the entered number is positive, negative, or equal to zero.
2. Write a programme that takes two integers from the user and displays the larger of the two.
3. Write a programme that asks the user to enter a grade on a scale from 1 to 5. The programme should display a description of the grade: fail (1), poor (2), satisfactory (3), good (4), very good (5). For a number outside the range, the programme should display an error message.
4. Write a programme that asks the user to enter three distinct integers and displays the smallest of them.
5. Write a programme that asks the user for a year and checks whether the given year is a leap year. A leap year is one that is divisible by 4 but not divisible by 100, unless it is also divisible by 400.
6. Write a programme that takes two integers from the user and displays whether the sum of both numbers is even or odd.
7. Write a programme that takes three floating-point numbers `a`, `b`, `c` from the user. Treat them as the coefficients of a quadratic equation $ax^2 + bx + c = 0$. Print all solution variants of this equation to standard output.
8. Write a programme that takes two integers as input from the user and uses the conditional operator to find and display the larger of the two.
9. Write a programme that takes three integers as input from the user and uses the conditional operator to find and display the smallest of them.
10. Using the conditional operator `?`, write a programme that takes a single integer from the user and displays “even” or “odd” depending on the value of the number.
11. Create a programme that asks the user to enter two floating-point numbers and uses the conditional operator to display which of them is larger, or whether they are equal to two decimal places.

12. Using the conditional operator, write a programme that asks the user to enter a grade on a scale from 0 to 100 and displays “Pass” if the grade is greater than or equal to 51, or “Fail” otherwise.
13. Write a programme that takes a year as input from the user and uses the conditional operator to check whether it is a leap year. The programme should display “Leap year” or “Not a leap year” depending on the result.

4 Bitwise operators

1. Swap the values of two integer variables without using an additional variable.
2. Check whether an integer is even or odd using bitwise operators.
3. Determine the value of the bit at a specified position in an integer.
4. Clear the value of the bit at a specified position in an integer.
5. Invert the values of all bits in an integer.
6. Shift the bits of an integer a specified number of positions to the left.
7. Shift the bits of an integer a specified number of positions to the right.
8. Determine the XOR of two integers.

5 Debugging - basics

1. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>

int main() {
    int a = 10; // a = , b =
    int b = 5;  // a = , b =
    a = a + b;  // a = , b =
    b = a - b;  // a = , b =
    a = a - b;  // a = , b =
    b = a * b;  // a = , b =
    a = b / a;  // a = , b =
    b = a << 2; // a = , b =
    a = b >> 1; // a = , b =
    b = a & b;  // a = , b =
    a = a ^ b;  // a = , b =
    b = ~a;     // a = , b =
    return 0;
}
```

2. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>

int main() {
    int a = 10; // a = , b = , c =
    int b = 5;  // a = , b = , c =
    int c = 0;  // a = , b = , c =
    a = a + b;  // a = , b = , c =
    b = a - b;  // a = , b = , c =
    a = a - b;  // a = , b = , c =
}
```

```

    c = a;      // a = , b = , c =
    a = b * c;  // a = , b = , c =
    b = a / c;  // a = , b = , c =
    c = b << 2;  // a = , b = , c =
    b = c >> 1;  // a = , b = , c =
    a = b & c;   // a = , b = , c =
    c = a ^ b;   // a = , b = , c =
    b = ~c;      // a = , b = , c =
    return 0;
}

```

3. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```

#include <stdio.h>
int main() {
    int x = 7;      // x = , y = , z =
    int y = 3;      // x = , y = , z =
    int z = 12;     // x = , y = , z =
    x = x * 2;      // x = , y = , z =
    y = y + x;      // x = , y = , z =
    z = z - y;      // x = , y = , z =
    x = z / 2;      // x = , y = , z =
    y = x % 3;      // x = , y = , z =
    z = x + y;      // x = , y = , z =
    x = z++;        // x = , y = , z =
    y = ++x;        // x = , y = , z =
    z = y--;        // x = , y = , z =
    return 0;
}

```

4. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```

#include <stdio.h>
int main() {
    int p = 15;     // p = , q = , r =
    int q = 6;      // p = , q = , r =
    int r = 1;      // p = , q = , r =
    p = p | q;      // p = , q = , r =
}

```

```

    q = p & 10;    // p = , q = , r =
    r = r << 3;    // p = , q = , r =
    p = p ^ r;     // p = , q = , r =
    q = q >> 1;    // p = , q = , r =
    r = ~p;        // p = , q = , r =
    p = q | r;     // p = , q = , r =
    q = p & r;     // p = , q = , r =
    r = r ^ q;     // p = , q = , r =
    return 0;
}

```

5. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```

#include <stdio.h>
int main() {
    float a = 10.5; // a = , b = , c =
    float b = 2.0;  // a = , b = , c =
    float c = 0.5;  // a = , b = , c =
    a = a / b;      // a = , b = , c =
    b = b * 3;      // a = , b = , c =
    c = a + b;      // a = , b = , c =
    a = c - a;      // a = , b = , c =
    b = b / 2;      // a = , b = , c =
    c = a * b;      // a = , b = , c =
    a = c + 0.5;    // a = , b = , c =
    b = a - c;      // a = , b = , c =
    c = a / b;      // a = , b = , c =
    return 0;
}

```

6. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```

#include <stdio.h>
int main() {
    int m = 20;    // m = , n = , k =
    int n = 4;     // m = , n = , k =
    int k = 2;     // m = , n = , k =
    m += n;        // m = , n = , k =
}

```

```
n -= k;          // m = , n = , k =  
k *= 3;          // m = , n = , k =  
m /= 2;          // m = , n = , k =  
n += m;          // m = , n = , k =  
k %= n;          // m = , n = , k =  
m <<= 1;         // m = , n = , k =  
n >>= 2;         // m = , n = , k =  
k ^= m;          // m = , n = , k =  
return 0;  
}
```

6 Loops

The tasks should be solved without arrays, strings, pointers, or built-in math functions. Do not create your own functions either.

1. Write a program that displays all integers from 1 to 100 using a **for** loop.
2. Create a program that asks the user to enter an integer **n**, and then displays the sum of all integers from 1 to **n** using a **while** loop.
3. Write a program that reads integers from the user until zero is entered, and then displays the sum of all positive numbers and the sum of all negative numbers, using a **do-while** loop.
4. Create a program that calculates and displays the factorial of a non-negative integer entered by the user, using a **for** loop.
5. Write a program that displays the first 10 numbers of the Fibonacci sequence using a **while** loop.
6. Write a program that asks the user to enter a positive integer **n**, and then calculates and displays $\lfloor \sqrt{n} \rfloor$ (the integer part/floor of the square root).
7. Write a program that asks the user to enter a positive integer **n**, and then calculates and displays $\lceil \sqrt{n} \rceil$ (the ceiling of the square root).
8. Write a program that asks the user to enter 10 positive integers and arranges them into a sequence $a_1, \dots, a_{10}$. Calculate and display how many elements of the sequence satisfy the inequality $a_k < \frac{a_{k-1} + a_{k+1}}{2}$ for $1 < k < 10$.
9. Write a program that asks the user to enter a positive integer n , and then n positive integers and arranges them into a sequence $a_1, \dots, a_n$. Calculate and display how many elements of the sequence satisfy the inequality $a_k < \frac{a_{k-1} + a_{k+1}}{2}$ for $1 < k < n$.
10. Write a program that checks the divisibility of a number **n** by all numbers from 2 to $n/2$. The program should print all divisors of that number. Read the value of **n** from the user.
11. Write a program that reads an integer **n** and prints all its digits from the end (starting from the units digit) and calculates their sum.
12. Write a program implementing the Euclidean algorithm in an iterative version using subtraction. The program should read two natural numbers **a** and **b**, and then print their greatest common divisor.

13. Write a program implementing the Euclidean algorithm in an iterative version using division. The program should read two natural numbers a and b , and then print their greatest common divisor.
14. Write a program that checks whether a given number n is a prime number. The program should check all potential divisors from 2 to the square root of n (use a loop to calculate the square root). Print an appropriate message.
15. Write a program simulating change-making. The program should read the amount to be given as change and print how many coins of denominations 5, 2, and 1 should be used to minimize the total number of coins. Print the number of coins of each type.
16. Write a program that determines the approximate value of the square root of a number a using Newton's method (iterative). The initial approximation is $x = a/2$, and subsequent values are calculated using the formula $x_{i+1} = (x_i + a/x_i)/2$. Perform 10 iterations of the algorithm and print the final result.
17. Write a program that reads from the user the degree of a polynomial, the coefficients (from the highest power to the constant term), and the value of x . Calculate the value of the polynomial using Horner's method, utilizing a loop. Display the result of the calculations.

7 Functions

Solve the exercises without using arrays, strings, pointers, or built-in math functions. You may create additional helper functions for each exercise. For each function described in the exercise, provide a test case (call it at least once with a valid argument). The exercises use the **camelCase** naming convention. Pay attention to whether the function should return a value or print it.

1. Write a function **sumTwoNumbers** that takes two integers as arguments. The function should print the sum of the numbers passed as arguments. Create a test case.
2. Write a function **calculateAbsoluteValue** that takes a floating-point number as an argument. The function should return the absolute value of the number passed as an argument. Create a test case.
3. Write a function **calculateFactorial** that takes a non-negative integer as an argument. The function should return the factorial of the number passed as an argument, computed using a non-recursive method. Create a test case.
4. Write a function **sumNumbers** that takes a positive integer n as an argument. The function should return the sum of numbers from 1 to n inclusive. Create a test case.
5. Write a function **sumSquares** that takes a positive integer n as an argument. The function should return the sum of squares of numbers from 1 to n inclusive. Create a test case.
6. Write a function **calculatePowerOfTwo** that takes an integer n as an argument. The function should return the value of 2^n . Create a test case.
7. Write a function **calculateSquareRootFloor** that takes a non-negative integer n as an argument. The function should return the integer part of the square root of n . Create a test case.
8. Write a function **countFunctionCalls** that takes no arguments. The function should count and print to standard output the number of times it has been called since the program started. Create a test case. Use static variables.
9. Write a function **calculateFactorialRecursively** that takes a non-negative integer n as an argument. The function should return the factorial of n , computed using a recursive method. Create a test case.

10. Write a function `calculateFibonacciRecursively` that takes a non-negative integer n as an argument. The function should return the n -th term of the Fibonacci sequence, computed using a recursive method. Create a test case.
11. Write a function `calculateArithmeticSequenceRecursively` that takes two positive integers as arguments: n (the index of the sequence term to compute) and d (the common difference of the arithmetic sequence), assuming the initial term a_1 equals 1. The function should return the n -th term of the arithmetic sequence, computed using a recursive method. Create a test case.
12. Write a function `calculateGeometricSequenceRecursively` that takes two positive integers as arguments: n (the index of the sequence term to compute) and d (the common ratio of the geometric sequence), assuming the initial term a_1 equals 1. The function should return the n -th term of the geometric sequence, computed using a recursive method. Create a test case.
13. Write a function `calculate13` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = 2f(n-1) + 3$, where $f(1) = 1$. Create a test case.
14. Write a function `calculate14` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = 3f(n-1) - 1$, where $f(1) = 2$. Create a test case.
15. Write a function `calculate15` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = f(n-1) + 2f(n-2)$, where $f(1) = 1$ and $f(2) = 2$. Create a test case.
16. Write a function `calculate16` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = 2f(n-1) + 3f(n-2)$, where $f(1) = 2$ and $f(2) = 3$. Create a test case.
17. Write a recursive function `calculate17` that takes a non-negative integer n as an argument. The function should return the value defined by $f(n) = f(0) + f(1) + \dots + f(n-1)$, where $f(0) = f(1) = 1$. Create a test case.
18. Write a recursive function `calculateGCD` that takes two positive integers n and m as arguments. The function should return the greatest common divisor (GCD) of these numbers using the Euclidean algorithm. Create a test case.

8 Pointers

The exercises should be solved without interpreting pointers as arrays.

1. Copy or retype the code and check the output on standard output:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("%zu\n", sizeof(int));
    printf("%zu\n", sizeof(int*));
    return 0;
}
```

2. In a single file, perform the following tasks:
 - Write a function `sum` with two arguments of type `int`. The function should return the sum of the values passed through the arguments. Create a test case.
 - Write a function `sumVals` with two arguments that are pointers to variables of type `int`. The function should return the sum of the values pointed to by these arguments. Create a test case.
 - Write a function `addPtr` with three arguments that are pointers to variables of type `int`. The function should be a procedure (return type `void`). The function should set the value of the variable pointed to by the third argument to the sum of the values pointed to by the first two arguments. Create a test case.
3. Write a function `copyInt` with the arguments: `x` of type `int` and `w`, which is a pointer to `int`. The function should copy the value of `x` to the variable pointed to by `w`. Create a test case.
4. Write a function `findMax` with two arguments: a pointer `num1` to a constant value of type `double` and a constant pointer `num2` to a variable of type `double`. The function should return the greater value among those pointed to by `num1` and `num2`. Create a test case.

5. Write a function `initInts` that takes no arguments and allocates a block of three variables of type `int`, setting their values to 5, -12, 33 respectively. The function returns a pointer to the middle variable. Create a test case in the `main` function that displays the values from the block created by the function.
6. Write a function `initFloats` that takes no arguments and allocates a block of three variables of type `float`, setting their values to 4.5, 2.3, -4.2 respectively. The function returns a pointer to the first variable in the block. Create a test case in the `main` function that displays the values from the block created by the function.
7. Write a function `initFlts` that takes no arguments and allocates a block of four variables of type `float`, setting their values to 0.5, 1.5, 2.5, and 3.5 respectively. The function returns a pointer to the last variable in the block. Create a test case in `main` to display the values from the block created by the function.
8. Write a function `sumToPtr` with three arguments: two pointers to constants of type `int` and a pointer to a variable of type `int`. The function should store the sum of the constant values pointed to by the first and second arguments into the variable pointed to by the third argument. Create a test case.
9. Write a function `sqrCopy` with two arguments: a pointer to a constant of type `int` and a pointer to a variable of type `int`. The function should copy the square of the constant's value to the variable pointed to by the second argument. Create a test case.
10. Write a function `subPtrs` with two arguments: a pointer `num1` to a constant value of type `double` and a constant pointer `num2` to a variable of type `double`. The function should return the difference of the values pointed to by `num1` and `num2`. Create a test case.
11. Write a function `sumSqrs` with two arguments: a pointer `num1` to a constant value of type `int` and a constant pointer `num2` to a variable of type `int`. The function should return the sum of the squares of the values pointed to by `num1` and `num2`. Create a test case.
12. Write a function `linFuncVal` with three arguments: a pointer `a` to a constant value of type `float`, a constant pointer `b` to a variable of type `float`, and a pointer `x` to a constant value of type `float`. The function should compute and return the value of the linear function $y=ax+b$ for argument `x`, where `a` and `b` are pointed to by the corresponding pointers. Create a test case.
13. Write a function `minPtr` with three arguments that are pointers to variables of type `int`. The function returns a pointer to the variable with the smallest value among those pointed to by the arguments. Create a test case in `main` to display the smallest value among three variables.
14. Write a function `multPtrs` with two arguments: a pointer `num1` to a constant value of type `double` and a constant pointer `num2` to a variable of type `double`. The function returns the product of the values pointed to by these pointers. Create a test case.

15. Write a function `absVal` with one argument, which is a pointer to a variable of type `int`. The function computes the absolute value of the variable pointed to by the pointer and updates that variable. Create a test case in `main` to display the value of the updated variable.
16. Write a function `swap` with two arguments: a pointer `ptr1` to a variable of type `int` and a pointer `ptr2` to another variable of the same type. The function swaps the values pointed to by the pointers. Create a test case.
17. Write a function `swapSign` with two arguments: a pointer `ptr1` to a variable of type `double` and a pointer `ptr2` to another variable of the same type. The function swaps the values pointed to by the pointers if they have different signs. Otherwise, it does nothing. Create a test case.
18. Write a function `swapIfGreater` that has two arguments. The first argument is a pointer `ptr1` to a variable of type `int`, and the second argument is a pointer `ptr2` to a variable of type `int`. The function should swap the values pointed to by these pointers, but only if the value pointed to by the first pointer is greater than the value pointed to by the second pointer. The function does not return any value. Create a test case for the function.
19. Write a function `swapMaxMiddle` that takes three pointers to variables of type `int` as arguments. The function should swap the values of the largest and the middle (in terms of value in non-decreasing order) variables pointed to by the pointers. Create a test case in `main` to display the values of the variables before and after the swap on the console.
20. Write a function `swapFirstPositive` that takes three pointers to variables of type `int` as arguments. The function should swap the values of the first and the first positive variable among those pointed to by the pointers. If there is no positive variable, the function should not make any changes. Create a test case in `main` to display the values of the variables before and after the swap on the console.
21. Write a function `swapOddEven` that takes four pointers to variables of type `int` as arguments. The function should swap the values of the variables so that all even values are at the beginning and odd values are at the end (the order within groups does not matter). Create a test case in `main` to display the values of the variables before and after the swap on the console.
22. Write a function `modifyThroughPtr` with two arguments: a pointer to a variable of type `int` and a value of type `int`. The function should perform the following operations: if the second value is positive — increase the value of the variable pointed to by the first argument by that value; if the second value is negative — decrease the value of the first variable by the absolute value of the second argument; and if the second value is 0 — do not change the value. The function returns the address of the modified variable. Create a test case.

9 Function Pointers

1. Write a function `calculate` that takes two arguments: a pointer to a function `operation` and an integer `number`. The function `operation` should take one argument of type `int` and return a value of type `int`. The function `calculate` should call the function `operation` with the argument `number` and return its result. Create a test case.
2. Write a function called `applyFunction` that takes three arguments: a pointer to a function `func` that takes one argument of type `int` and returns `int`, and two integers: `start` and `end`. The function `applyFunction` should call the function `func` for each number in the range from `start` to `end` (inclusive) and print the results to standard output. Create a test case.
3. Write a function that takes three arguments:

- two pointers to functions with one argument of type `int` returning a value of type `int`,
- a positive value `n` of type `int`,

and returns 1 if the functions received as arguments have the same sign for integer values from 0 to `n`, and returns 0 otherwise. Create a test case.

4. Write a function `calculateOperation` that takes as arguments: a pointer to a function `operation` that takes two arguments of type `double` and returns `double`, and two arguments of type `double` — `number1` and `number2`. The function `calculateOperation` should return the result of calling the function `operation` on the arguments `number1` and `number2`. Create a test case.
5. Write a function `modifyAndSum` that takes as an argument a pointer to a function `modifier` that takes one argument of type `int` and returns `int`, and two arguments of type `int`: `number1` and `number2`. The function `modifyAndSum` should modify both numbers using the function `modifier` and return their sum. An example of a `modifier` function could be a function that increments a number by 1 or changes the sign of a number. Create a test case.
6. Create a function `applyCondition` that takes three arguments: a pointer to a function `condition` returning a value of type `int` and taking an `int`, a pointer to a function `action` also taking and returning `int`, and an integer value `value`. The function `applyCondition` should first call `condition` with `value` as the argument. If the result is 1, `applyCondition` should then call `action` on `value` and return the result. Otherwise, it should return `value` unchanged. Create a test case.

7. Create a function `executeSequence` that takes two arguments: a pointer to a function `operation` returning `void` and taking an `int`, and an integer `count`. The function `executeSequence` should call the function `operation` exactly `count` times, passing consecutive numbers from 1 to `count` as the argument. Create a test case.
8. Create a function `modifyValue` that takes an argument of type `int` and a pointer to a function `modifier` that takes no arguments but returns a value of type `int`. The function `modifyValue` should add the result of calling the function `modifier` to the passed value and return the resulting sum. Create a test case.
9. Create a function `processPair` that takes two integers and a pointer to a function `processor` that returns `double` and takes two arguments of type `int`. The function `processPair` should call `processor` with the passed numbers, round the result down to the nearest integer, and return that value as `int`. Create a test case.
10. Create a function `transformLoop` that takes a pointer to a function `transform` returning `int` and taking `int`, an initial value `start`, and a number of iterations `iterations`. The function should perform the specified number of iterations, in each iteration assigning to the result variable the value returned by the function `transform` called on the result from the previous iteration (on the value `start` in the first iteration). At the end, the function should return the final value. Create a test case.
11. Create a function `computeWithFallback` that takes three arguments: a pointer to a function `primary` taking two parameters of type `int` and returning `int`, a pointer to a function `backup` with the same signature, and two integer values `a` and `b`. The function `computeWithFallback` should call the function `primary` with the arguments `a` and `b`. If the result of the function `primary` is less than zero, the function `computeWithFallback` should call `backup` with the same arguments `a` and `b` and return the result of that function. Otherwise (when the result of the function `primary` is greater than or equal to zero), the function `computeWithFallback` should return the result of the function `primary`. The function `computeWithFallback` should return a value of type `int`. Create a test case.

10 One-Dimensional Arrays

The exercises should be solved without converting numbers to strings. In the exercises, do not use double pointers or equivalents of multi-dimensional arrays — the syntax should use a single dimension only. In the exercises, do not consider an array of size zero, as size-related arguments are positive.

1. Write a function `findMaxValue` that takes as arguments an integer array `numbers` and its size `size` (a positive integer). The function should search the array and display the maximum value found in it. Create a test case.
2. Write a function `average` whose arguments are a positive integer `n` and an `n`-element array `tab` of elements of type `int`, and returns the arithmetic mean as a floating-point number. Create a test case.
3. Write a function `sumSquares` whose arguments are a positive integer `n` and an `n`-element array `tab` of elements of type `int`, and returns the sum of squares of the array elements. Create a test case.
4. Write a function `copyArr` that takes three arguments: a positive integer `n` and two `n`-element arrays `tab1`, `tab2` of elements of type `int`, and copies the elements of array `tab1` to array `tab2` in order. Create a test case.
5. Write a function `revCopy` that takes three arguments: a positive integer `n` and two `n`-element arrays `tab1`, `tab2` of elements of type `int`, and copies the elements of array `tab1` to array `tab2` in reverse order. Create a test case.
6. Write a function `reverseArr` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should reverse the order of elements in the array `tab` directly in place, without using an additional array to store results. Create a test case.
7. Write a function `maxValue` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should search the array and return the largest value found in it. Assume the array is not empty. Create a test case.
8. Write a function `minValue` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should search the array and return the smallest value found in it. Assume the array is not empty. Create a test case.

9. Write a function `maxIdx` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should find and return the index of the largest element in the array. If more than one element has the maximum value, the function should return the index of the first one (i.e., the smallest possible index). Assume the array is not empty. Create a test case.
10. Write a function `minIdxMax` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should search the array and return the index of the smallest element in the array. If there are multiple identical smallest elements, return the largest possible index among them. Assume the array is not empty. Create a test case.
11. Write a function `shiftLeft` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `float`. The function should shift all elements of the array one position to the left, and the last element in the array should be replaced by the first element of the original array. Assume the array is not empty. Create a test case.
12. Write a function `shiftRight2` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should shift all elements of the array two positions to the right. Elements that “fall off” the end of the array should appear at its beginning in the same order. Create a test case.
13. Write a function `shiftLeft` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should return a pointer to a newly created dynamic array in which all elements have been shifted one position to the left relative to the original array, and the last element of the new array equals the first element of the original array. Create a test case.
14. Write a function `shiftRight` whose arguments are a positive integer `n` representing the array size and an `n`-element array `tab` of elements of type `int`. The function should return a pointer to a newly created dynamic array in which all elements have been shifted one position to the right relative to the original array, and the first element of the new array equals the last element of the original array. Create a test case.
15. Below is code in the C language. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each specific line of code is executed. Adding new lines or disrupting the code structure means incorrect completion of the task.

```
#include <stdio.h>
#include <stdlib.h>

int foo(int*a,int*b)
{
    return *a-*b;
```

```

}

int main()
{
    int tab[] = {3,3,0,0,3,3,-4,-4,0,1};
    int *wsk=tab-1;
    int b = *(wsk+=4); //b=
    int c = b+3; // b= , c=
    int d = foo(&b,&c); // b= , c= , d=
    int e = (wsk+--1)[4]; // b= , c= , d= , e=
    e = (d *= 2) + (c += 2); // b= , c= , d= , e=
    c = d + (b+=4); // b= , c= , d= , e=
    e = (--c)-(d++); // b= , c= , d= , e=
    b = *wsk + e; // b= , c= , d= , e=
    return 0;
}

```

16. Below is code in the C language. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each specific line of code is executed. Adding new lines or disrupting the code structure means incorrect completion of the task.

```

#include <stdio.h>
#include <stdlib.h>

int foo(int*a,int*b)
{
    return *a-*b;
}

int main()
{
    int tab[] = {1,5,5,2,3,3,-4,-4,0,1};
    int *wsk=tab-1;
    int b = *(wsk+=4); //b=
    int c = b+3; // b= , c=
    int d = foo(&b,&c); // b= , c= , d=
    int e = (wsk+--1)[4]; // b= , c= , d= , e=
    e = (d -= 2) - (c += 2); // b= , c= , d= , e=
    c = d + (b+=4); // b= , c= , d= , e=
    e = (--c)-(d++); // b= , c= , d= , e=
    b = *wsk + e; // b= , c= , d= , e=
}

```

```
    return 0;  
}
```

17. Write a function called `findElement` that takes as arguments an integer array, the array size, an integer `val` to find, and a pointer to a function `isEqual`. The function `isEqual` takes two integers as arguments and returns a value of type 0 or 1, indicating whether the numbers are equal, respectively. The function `findElement` should search the array to find the value `val` according to the rules defined by the function `isEqual`, and then return the index of that element in the array (the first one encountered) or `-1` if the element does not exist. Create a test case for the function `findElement`.
18. Write a function `findWithCondition` that takes as arguments an integer array, the array size, and a pointer to a function `condition`. The function `condition` takes an integer as an argument and returns a value of type 0 or 1, indicating whether the number satisfies a specified condition. The function `findWithCondition` should search the array to find the last number satisfying the condition defined by the function `condition`, and then return the index of that element in the array or `-1` if no such element exists. Create a test case for the function `findWithCondition`.
19. Write a function `isSymmetric` that takes two arguments: a positive integer `n` and an `n`-element array `tab` of elements of type `int`. The function should return the value 1 if the array `tab` is symmetric (i.e., the first element equals the last, the next equals the second-to-last, etc.) and return zero otherwise. Create a test case for the function.
20. Write a function `areArraysEqual` that takes three arguments: a positive integer `n` and two `n`-element arrays `tab1` and `tab2` of elements of type `int`. The function should check whether both arrays are identical. If so, the function should return 1; otherwise, return 0. Create a test case for the function.
21. Write a function `indexOfMaxAbs` that takes two arguments: a positive integer `n` and an `n`-element array `tab` of elements of type `int`, and returns the index of the first occurrence of the largest absolute value among the elements stored in array `tab`. Create a test case for the function.
22. Write a function `rotateLeftByTwo` that takes two arguments: a positive integer `n` ($n > 3$) and an `n`-element array `tab` of elements of type `double`, and shifts all elements of the array two positions to the left (so that the value of the element at index `n-1` ends up at index `n-3`, the value at index `n-2` ends up at index `n-4`, while the value at index 0 goes to index `n-2`, and the value at index 1 goes to index `n-1`). Create a test case for the function.
23. Write a function `multiplyElementwise` that takes four arguments: a positive integer `n`, `n`-element arrays `tab1` and `tab2`, and an `n`-element array `tab3` of elements of type `double`. The function should compute the product of the elements of arrays `tab1` and `tab2` at the same indices and store the results in array `tab3`. Create a test case for the function.

24. Write a function `sumOddIndices` that takes two arguments: a positive integer n and an n -element array `tab` of elements of type `int`. The function should return the sum of the elements located at odd indices. Create a test case.
25. Write a function `countPositiveElements` that takes two arguments: a positive integer n and an n -element array `tab` of elements of type `int`. The function should return the number of positive elements in the array. Create a test case.

Example. For an array containing the elements 3,-4,5 the function should return 2.

26. Write a function `doubleOddElements` that takes two arguments: a positive integer n and an n -element array `tab` of elements of type `int`. The function should double the odd-valued elements in the array. Create a test case.
27. Write a function `increaseByIndex` that takes two arguments: a positive integer n and an n -element array `tab` of elements of type `int`. The function should increase the value of each array element by its index. Create a test case.
28. Write a function `multiplyElementwise` that takes four arguments: a positive integer n , n -element arrays `tab1` and `tab2`, and an n -element array `tab3` of elements of type `double`. The function should compute the product of the elements of arrays `tab1` and `tab2` at the same indices and store the results in array `tab3`. Create a test case for the function.
29. Write a function `sumAboveThreshold` that takes three arguments: an integer k , a positive integer n , and an n -element array `arr` of elements of type `int`. The function should return the sum of all elements of array `arr` that are greater than the number k . Create two test cases. Example: for $k=10$, $n=6$ and the array 4,12,3,15,8,25 the function should return 52.
30. Write a function `findFirstDivisibleIndex` that takes three arguments: a positive integer p , a positive integer n , and an n -element array `numbers` of elements of type `int`. The function should return the index of the first occurrence of a number divisible by p in the array `numbers`. If no element is divisible by p , the function should return -1. Create two test cases. Example: for $p=3$, $n=7$ and the array 5,11,9,2,4,7,6 the function should return 2.
31. Write a function `countInRange` that takes three arguments: an integer a , an integer b , a positive integer n , and an n -element array `data` of elements of type `int`. The function should count how many elements of array `data` fall within the interval $[a,b]$ (including a and b). Create two test cases. Example: for $a=5$, $b=15$, $n=8$ and the array 3,7,22,10,5,18,9,4 the function should return 4.
32. Write a function `sumDivisibleBy3` that takes two arguments: a positive integer n and an n -element array `tab` of elements of type `int`. The function should return the sum of elements divisible by 3, and zero if there are no such elements. Create a test case. Example: for the array 7,9,2,6,5 the function should return 15.

33. Write a function **productEven** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function should return the product of even-valued elements, and 1 if there are no such elements. Create a test case. Example: for the array 3,4,5,2,7 the function should return 8.
34. Write a function **countNegInd** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function should return the count of negative elements that are located at even indices of the array. If there are no such elements, the function should return 0. Create a test case. Example: for the array -2,4,-5,-7,6 the function should return 2.
35. Write a function **minArrays** that takes four arguments: a positive integer **n**, **n**-element arrays **tabA**, **tabB**, and **tabC** of elements of type **int**. The function should select the minimum value from the elements of arrays **tabA** and **tabB** at the same indices and store the results in array **tabC**. Create a test case. Example: **tabA** = {7,-2,14}, **tabB** = {9,6,-3} → **tabC** = {7,-2,-3}.
36. Write a function **sumArrays** that takes four arguments: a positive integer **m**, **m**-element arrays **array1**, **array2**, and **arrayResult** of elements of type **double**. The function should sum the corresponding elements of arrays **array1** and **array2**, and then store the results in array **arrayResult**. Create a test case.
37. Write a function **sumOdd** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function should return the sum of odd-valued elements, and 0 if there are no such elements. Create a test case. Example: for the array 2,7,4,9,3 the function should return 19.
38. Write a function **multiplyByPosition** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function modifies the array by multiplying each element by its index plus 1 (the element at index 0 is multiplied by 1, the element at index 1 by 2, etc.). Create a test case. Example: array {2,3,4} → {2,6,12}.
39. Write a function **decreaseBySquare** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function should decrease the value of each array element by the square of its index. Create a test case. Example: array {5,10,15,20} → {5,9,11,11}.
40. Write a function **replaceWithPower** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function should replace the value of each array element with that element raised to the power equal to its index. For index 0, the element remains unchanged. Create a test case. Example: array {2,3,4,-3,-2} → {2,3,16,-27,16}.
41. Write a function **sumOddIndex** that takes two arguments: a positive integer **n** and an **n**-element array **tab** of elements of type **int**. The function should return the sum of elements at odd positions, and zero if there are no such elements. Create a test case.

Example: for the array 3,-4,5,7,3 the function should return 3, because at odd indices there are -4 and 7.

42. Write a function `countMatching` that takes as arguments an integer array, the array size, and a pointer to a function `predicate`. The function `predicate` takes an integer as an argument and returns a value of 0 or 1, indicating whether the number satisfies a specified condition. The function `countMatching` should count and return the number of elements in the array that satisfy the condition. Create a test case. Example: for a function $f(x)=1$ for even numbers, $f(x)=0$ for odd numbers, and the array {2,7,8,3,10} the function should return 3.
43. Write a function `productPositive` that takes two arguments: a positive integer `n` and an `n`-element array `tab` of elements of type `int`. The function should return the product of all positive elements in the array. If the array contains no positive elements, the function should return 1. Create a test case. Example: for the array {3,-2,5,0,-1,4} the function should return 60.
44. Write a function `countDivisible` that takes three arguments: a positive integer `n`, an `n`-element array `tab` of elements of type `int`, and a positive integer `divisor`. The function should return the count of array elements that are divisible by `divisor`. Create a test case. Example: for the array {12,7,-9,15,4,21} and `divisor=3` the function should return 4.
45. Write a function `replaceWithFactorial` that takes two arguments: a positive integer `n` and an `n`-element array `tab` of elements of type `int`. The function should replace the value of each array element with the factorial of the absolute value of that element. For values greater than 12, the element should be replaced with the value -1. Create a test case. Example: array {3,-4,0,5,15} $\rightarrow$ {6,24,1,120,-1}.
46. Write a function `getMaximumValue` that takes as arguments a non-empty integer array, the array size, and a pointer to a function `isValid`. The function `isValid` takes an integer and returns 0 or 1. The function `getMaximumValue` should find the largest value among the array elements that satisfy the condition and return it. If no element satisfies the condition, the function should return 0. Create a test case. Example: for $f(x)=1$ when $x\%3==0$ and the array {7,9,4,12,6} the function should return 12.
47. Write a function `sumOfDivisors` that takes two arguments: a positive integer `n` and an `n`-element array `tab` of elements of type `int`. The function should replace the value of each array element with the sum of all positive divisors of the absolute value of that element (including the number itself). Create a test case. Example: array {6,-12,1,7,-8} $\rightarrow$ {12,28,1,8,15}.
48. Write a function `sumEvenIndices` that takes two arguments: a positive integer `n` and an `n`-element array `tab` of elements of type `int`. The function should return the sum of elements located at even index positions (0, 2, 4, ...). Create a test case. Example: for the array {10,3,7,-2,5,8} the function should return 22.

49. Write a function `multiplyByDigitSum` that takes two arguments: a positive integer n and an n -element array `tab` of elements of type `int`. The function should replace the value of each array element with the product of that value and the sum of the digits of the absolute value of that element. Create a test case. Example: array `{12,-34,5,100,-7}` $\rightarrow$ `{36,-238,25,100,-49}`.

11 Strings

1. Write a function `length` that takes a string as an argument and returns its length as a value. Use the `char` type and do not use any built-in functions other than input/output operations. Create a test case.
2. Write a function `countNums` that takes a `char` array as an argument and returns the number of characters in this array that are digits. Do not use any built-in language functions other than input/output operations. Create a test case for your function.
3. Write a function `countVow` that takes a `char` array as an argument and returns the number of characters in this array that are vowels of the Latin alphabet. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
4. Write a function `cmpStrNew` that takes two arguments of type `char[]` (character arrays) and returns 1 if the strings are identical, and 0 otherwise. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
5. Write a function `lexComp` that takes two arguments of type `char[]` (character arrays) and returns 1 if the first string comes earlier in lexicographic order than the second one, and 0 otherwise. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
6. Write a function `toLowerNew` that takes a `char` array as an argument and converts all uppercase letters in it to lowercase. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
7. Write a function `toUpperNew` that takes a `char` array as an argument and converts all lowercase letters in it to uppercase. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
8. Write a function `strCopyNew` that takes two arguments of type `char[]` (character arrays): a source and a destination. The function copies the string located in the source array into the destination array. Assume that there is enough space in the destination array. Do not use any built-in language functions other than input/output operations. Create a test case for this function.

9. Write a function **strNcopy** that takes three arguments: two character arrays **nap1**, **nap2** and a positive integer **n**. The function copies the first **n** characters from the string stored in array **nap1** to array **nap2**. If the string in array **nap1** is shorter than **n** characters, the function copies the entire string. The function guarantees that a null-terminating character (`'\0'`) will be placed at the end of the string in array **nap2**. Assume that there is enough space in array **nap2**. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
10. Write a function **strConcat** that takes three character arrays as arguments: **nap1**, **nap2** and **nap3**. The function writes to array **nap3** the concatenation (joining) of the strings located in arrays **nap1** and **nap2**. Assume that there is enough space in array **nap3**. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
11. Write a function **rmLower** that takes a **char** array as an argument and removes from it all characters that are lowercase letters. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
12. Write a function **rmDigits** that takes a **char** array as an argument and removes from it all characters that are digits. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
13. Write a function **cutStr** that takes as arguments a **char** array and two integers **n** and **m**, and cuts out from the received string the characters with indices from **n** to **m** (assuming that $n < m$). Do not use any built-in language functions other than input/output operations. Create a test case for this function.
14. Write a function **wStrCopyNew** that takes two arguments of type **wchar_t[]** (character arrays): a source and a destination. The function copies the string located in the source array into the destination array. Assume that there is enough space in the destination array. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
15. Write a function **wStrConcat** that takes three character arrays as arguments: **nap1**, **nap2** and **nap3** (use the **wchar_t** character type). The function writes to array **nap3** the concatenation (joining) of the strings located in arrays **nap1** and **nap2**. Assume that there is enough space in array **nap3**. Do not use any built-in language functions other than input/output operations. Create a test case for this function.
16. Write a function **w16** whose argument is a string. If the string contains characters other than digits, the function should return zero. If the string contains only digits, the function should return an integer formed by sequentially transcribing the digit characters. Assume that the string is exactly 3 characters long. In this task, do not use library functions other than input/output instructions. Create a test case. Use the **wchar_t** type in your solution.

17. Write a function `rmOdd` whose argument is a string. The function is supposed to remove characters from the string at odd indices. Create a test case.
18. Write a function `copyEveryThird` whose argument is a string. The function is supposed to return a new string that contains only every third character of the original string. Create a test case.

Example: For the string "ABCXYZ" the function should return "AX".

19. Write a function `indexLower` whose argument is a string. The function returns the index number at which the first lowercase letter from the left occurs. In the case of an empty string or no lowercase letters, the function should return zero. In this task, do not use library functions other than input/output instructions. Create a test case.
20. Write a function `countOdds` whose argument is a string. The function should return the number of odd digit characters appearing in the string. Create a test case.
21. Write a function `toLowerNew` that takes a pointer to a string of type `const char*` as an argument and returns a pointer to a new string in which all uppercase letters have been converted to lowercase. The original character array remains unchanged. Remember to allocate memory for the new string. Do not use any built-in language functions other than input/output operations and memory allocation functions. Create a test case for this function.
22. Write a function `toUpperNew` that takes a pointer to a string of type `const char*` as an argument and returns a pointer to a new string in which all lowercase letters have been converted to uppercase. The original character array remains unchanged. Remember to allocate memory for the new string. Do not use any built-in language functions other than input/output operations and memory allocation functions. Create a test case for this function.

12 Multidimensional arrays

1. Write a function `sumMatrix` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of `int` elements with dimensions $n \times m$ and returns as its value the sum of the array's elements. Create a test case for the function.
2. Write a function `sumArray` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of arrays of `int` elements with dimensions $n \times m$ and returns as its value the sum of the array's elements. Create a test case for the function.
3. Write a function `maxRowIdx` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of `int` elements with dimensions $n \times m$. The function should return the index of the row containing the largest element (in case of several possible indices, return the smallest one). Create a test case for the function.
4. Write a function `minColIdx` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of arrays of `int` elements with dimensions $n \times m$. The function should return the index of the column containing the smallest element (in case of several possible indices, return the smallest one). Create a test case for the function.
5. Write a function `copyMat` that takes as arguments two positive integers `n` and `m` and two two-dimensional arrays of `int` elements with dimensions $n \times m$. The function should copy the contents of the second array into the first array. Create a test case for the function.
6. Write a function `copyArr2D` that takes as arguments two positive integers `n` and `m` and two two-dimensional arrays of arrays of `int` elements with dimensions $n \times m$. The function should copy the contents of the second array into the first array. Create a test case for the function.
7. Write a function `swapElems` that takes as arguments two positive integers `n` and `m` and two two-dimensional arrays of `int` elements with dimensions $n \times m$. The function should swap the corresponding elements of both arrays with each other. Create a test case for the function.
8. Write a function `swapItems` that takes as arguments two positive integers `n` and `m` and two two-dimensional arrays of arrays of `int` elements with dimensions $n \times m$. The function should swap the corresponding elements of both arrays with each other. Create a test case for the function.

9. Write a function **revRows** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of **int** elements with dimensions $n \times m$. The function should reverse the order of elements in each row. Create a test case for the function.
10. Write a function **revRowArr** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should reverse the order of elements in each row. Create a test case for the function.
11. Write a function **revCols** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of **int** elements with dimensions $n \times m$. The function should reverse the order of elements in each column. Create a test case for the function.
12. Write a function **revColArr** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should reverse the order of elements in each column. Create a test case for the function.
13. Write a function **transpose** that takes as arguments a positive integer **n** and a two-dimensional square array of **int** elements with dimensions $n \times n$. The function should transpose the array's elements (swap rows with columns). Create a test case for the function.
14. Write a function **transArr** that takes as arguments a positive integer **n** and a two-dimensional square array of arrays of **int** elements with dimensions $n \times n$. The function should transpose the array's elements (swap rows with columns). Create a test case for the function.
15. Write a function **oddAvg** whose argument is a two-dimensional array of arrays (containing variables of type **int**) and its dimensions $n, n > 1$ and $m, m > 1$. The function should return the average of the elements at odd indices (both indices must be odd simultaneously). Create a test case.
16. Write a function **evenSum** whose argument is a two-dimensional array of arrays (containing variables of type **int**) and its dimensions $n, n > 1$ and $m, m > 1$. The function should return the sum of the elements at even indices (both row and column indices are even). Create a test case.
17. Write a function **swap2nd** whose argument is a two-dimensional array of arrays (containing variables of type **int**) and its dimensions n and m . The function should swap the row at index 1 with the second-to-last row. If the array has fewer than four rows, the function should do nothing. Create a test case.

Example:

$$\begin{bmatrix} 2 & 3 & -3 \\ 1 & 4 & 7 \\ -3 & -6 & 11 \\ -2 & 8 & 23 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 3 & -3 \\ -3 & -6 & 11 \\ 1 & 4 & 7 \\ -2 & 8 & 23 \end{bmatrix}$$

18. Write a function `flipDiag` whose arguments are two positive integers n and m and a two-dimensional array of elements (containing variables of type `int`) of dimension $n \times m$. The function should reverse the order of elements lying on the main diagonal. The function should work only for square matrices; otherwise it should do nothing. Create a test case.

Example:

$$\begin{bmatrix} 2 & 3 & -3 \\ 1 & 4 & 7 \\ -3 & -6 & 11 \end{bmatrix} \rightarrow \begin{bmatrix} 11 & 3 & -3 \\ 1 & 4 & 7 \\ -3 & -6 & 2 \end{bmatrix}$$

19. Write a function `swapColumns` that takes as arguments a two-dimensional array of arrays of integers, its dimensions, and two column indices to swap. The function should swap the indicated columns and return the modified array. Handle the situation when the given indices are invalid — in that case the function should do nothing. Create a test case for the function.
20. Write a function `revOddCol` whose arguments are two positive integers n and m and a two-dimensional array of elements (containing variables of type `int`) of dimension $n \times m$. The function should reverse the order of elements in columns with odd indices. Create a test case.

Example:

$$\begin{bmatrix} 2 & 3 & -3 \\ 1 & 4 & 7 \\ -3 & -6 & 11 \\ -2 & 8 & 23 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 8 & -3 \\ 1 & -6 & 7 \\ -3 & 4 & 11 \\ -2 & 3 & 23 \end{bmatrix}$$

21. Create a function `idxDiff` whose argument is a two-dimensional square array of arrays (containing elements of type `int`) and its dimension $n, n > 0$. The function should return the difference between the sum of indices of the smallest element and the sum of indices of the largest element in the array. In case of several elements with the smallest or largest value, these should be the smallest possible sums of indices. Create a test case.
22. Write a function `sqEndCols` whose argument is a two-dimensional array of arrays (containing variables of type `int`) and its dimensions n and m ($n > 0, m > 1$). The function should square the elements in the first and last columns. Create a test case.

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 5 & 8 \\ 9 & 8 & -2 \\ -2 & 3 & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 2 & 9 \\ 0 & 5 & 64 \\ 81 & 8 & 4 \\ 4 & 3 & 16 \end{bmatrix}$$

23. Write a function **transCopy** that takes as arguments a positive integer **n** and a two-dimensional square array of arrays of **int** elements with dimensions $n \times n$. The function should return a pointer to a newly-created dynamic two-dimensional array created from the passed argument by transposing its elements. Create a test case for the function.
24. Write a function **newArrOneD** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should return a pointer to a newly-created dynamic one-dimensional array created from the argument by copying the elements row by row. Create a test case for the function.
25. Write a function **sortRows** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should sort the elements of each row separately from smallest to largest. Create a test case for the function.

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -5 & 8 \\ 9 & 8 & -2 \\ -3 & 5 & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 2 & 3 \\ -5 & 0 & 8 \\ -2 & 8 & 9 \\ -3 & 4 & 5 \end{bmatrix}$$

26. Write a function **sortCols** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should sort the elements of each column separately from largest to smallest. Create a test case for the function.

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -5 & 8 \\ 9 & 8 & -2 \\ -3 & 5 & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 9 & 8 & 8 \\ 1 & 5 & 4 \\ 0 & 2 & 3 \\ -3 & -5 & -2 \end{bmatrix}$$

27. Write a function **showRows** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should display the array's elements in the following way: the elements of each row should be separated by a comma, and consecutive rows should be separated by a newline

character; there should be no comma at the end of each line. Create a test case for the function.

Example: For the array

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -5 & 8 \\ 9 & 8 & -2 \\ -3 & 5 & 4 \end{bmatrix}$$

it should display exactly:

```
1,2,3
0,-5,8
9,8,-2
-3,5,4
```

28. Write a function `showCols` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of arrays of `int` elements with dimensions $n \times m$. The function should display the array's elements in the following way: the elements of each column should be separated by a comma on a single line, and consecutive columns should be separated by a newline character; there should be no comma at the end of each line. Create a test case for the function.

Example: For the array

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -5 & 8 \\ 9 & 8 & -2 \\ -3 & 5 & 4 \end{bmatrix}$$

it should display exactly:

```
1,0,9,-3
2,-5,8,5
3,8,-2,4
```

29. Write a function `snkPrint` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of arrays of `int` elements with dimensions $n \times m$. The function should display the array's elements in a single row separated by spaces in a “snail/snake” pattern (see the example). Create a test case for the function.

Example: For the array

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -5 & 8 \\ 9 & 8 & -2 \\ -3 & 5 & 4 \end{bmatrix}$$

it should display exactly:

```
1 2 3 8 -2 4 5 -3 9 0 -5 8
```

30. Write a function `rotateRight` that takes as arguments two positive integers `n` and `m` and a two-dimensional array of arrays of `int` elements with dimensions $n \times m$. The function should return a pointer to a newly-created dynamic two-dimensional array with dimensions $m \times n$ created from the passed argument by rotating it 90 degrees to the right (clockwise). Create a test case for the function.

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -5 & 8 \\ 9 & 8 & -2 \\ -3 & 5 & 4 \end{bmatrix} \rightarrow \begin{bmatrix} -3 & 9 & 0 & 1 \\ 5 & 8 & -5 & 2 \\ 4 & -2 & 8 & 3 \end{bmatrix}$$

31. Write a function `mulMat` that takes as arguments three positive integers `n`, `m` and `p` and two two-dimensional arrays of arrays of `int` elements: the first with dimensions $n \times m$ and the second with dimensions $m \times p$. The function should return a pointer to a newly-created dynamic two-dimensional array with dimensions $n \times p$ being the matrix product of both arrays. Create a test case for the function.

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 & 1 \\ 1 & -1 & 3 \end{bmatrix} = \begin{bmatrix} 4 & -2 & 7 \\ 10 & -4 & 15 \\ -2 & 0 & -1 \end{bmatrix}$$

32. Write a function `subMat` that takes as arguments two positive integers `n` and `m`, a two-dimensional array of arrays of `int` elements with dimensions $n \times m$, and two non-negative integers `r` and `c` ($0 \leq r < n$, $0 \leq c < m$). The function should return a pointer to a newly-created dynamic two-dimensional array with dimensions $(n - 1) \times (m - 1)$ created from the passed array by removing the row at index `r` and the column at index `c`. Create a test case for the function.

Example (for $r = 1$, $c = 2$):

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & -5 & 8 & 6 \\ 9 & 8 & -2 & 7 \\ -3 & 5 & 4 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 2 & 4 \\ 9 & 8 & 7 \\ -3 & 5 & 1 \end{bmatrix}$$

33. Write a function **mergeHoriz** that takes as arguments three positive integers **n**, **m1** and **m2** and two two-dimensional arrays of arrays of **int** elements: the first with dimensions $n \times m_1$ and the second with dimensions $n \times m_2$. The function should return a pointer to a newly-created dynamic two-dimensional array with dimensions $n \times (m_1 + m_2)$ created by joining both arrays horizontally (the second array placed to the right of the first). Create a test case for the function.

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, \quad \begin{bmatrix} -1 & 0 & 7 \\ 8 & 9 & -2 \\ 3 & 3 & 3 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 2 & -1 & 0 & 7 \\ 3 & 4 & 8 & 9 & -2 \\ 5 & 6 & 3 & 3 & 3 \end{bmatrix}$$

34. Write a function **borderFrame** that takes as arguments two positive integers **n** and **m** and a two-dimensional array of arrays of **int** elements with dimensions $n \times m$. The function should return a pointer to a newly-created dynamic two-dimensional array with dimensions $(n + 2) \times (m + 2)$ created by surrounding the passed array with a frame of zeros. Create a test case for the function.

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \longrightarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

13 Complex data types

In the following exercises, unless otherwise stated, the terms name and alias of a structure can be used interchangeably.

1. Define a structure **Car** with fields: **brand** (string), **model** (character array of size 20), **year** (integer) and **isAvailable** (value 0 or 1), which represents information about a car's availability for sale. In **main**, create several variables of type **Car**.
2. Create a structure **Student** that contains: **firstName** (character string), **lastName** (character string), **id** (integer) and **gpa** (floating-point number), representing a student's grade point average. In **main**, create three variables of type **Student**. Then build an array from these three variables.
3. Plan a structure **Rectangle** having fields: **width** (floating-point number), **height** (floating-point number) and **color** (character string), which is to be used for storing information about a rectangle's dimensions and color. In **main**, create a 4-element array of variables of type **Rectangle**.
4. Create a structure **WeatherInfo** containing: **temperature** (floating-point number), **humidity** (floating-point number) and **windSpeed** (floating-point number), which will be used to store meteorological data. In **main**, create several variables of type **WeatherInfo**.
5. Design a structure **Book** with fields: **title** (character string), **author** (character string), **publishedYear** (integer) and **pages** (integer), which is to describe basic information about a book. In **main**, create a 5-element array of variables of type **Book**.
6. Write a structure **Person** with fields **name** (character array of length 20) and **age** (of type **int**). Then write the functions and call each of them at least once:
 - a) **initPerson** - a function that takes two arguments, name and age, and returns a pointer to a newly-created structure setting its members from the passed arguments.
 - b) **showPerson** - a function whose argument is a variable of type **Person**. The function should print a description of the passed argument (write the age and name to standard output).
 - c) **birthday** - a function whose argument is a pointer to a structure of type **Person**. The function should increase the age by 1 in the passed argument.

Create a test case for each of the functions.

7. Write a structure **Car** with fields **brand** (character array of length 20) and **mileage** (of type **int**). Then write the functions and call each of them at least once:
- a) **initCar** - a function that takes two arguments: **brand** and **mileage**, and returns a newly-created structure setting its members from the passed arguments.
 - b) **showCar** - a function whose argument is a variable of type **Car**. The function should print a description of the passed argument (print the **brand** and **mileage** to standard output).
 - c) **mileageService** - a function whose argument is a pointer to a structure of type **Car**. The function should add 10000 to the **mileage** in the passed argument.

Create a test case for each of the functions.

8. Write a structure **Book** with fields **title** (character array of length 50) and **page_count** (of type **int**). Then write the functions and call each of them at least once:
- a) **initBook** - a function that takes two arguments: **title** and number of pages, and returns a newly-created structure setting its members from the passed arguments.
 - b) **showBook** - a function whose argument is a variable of type **Book**. The function should print a description of the passed argument (print the **title** and number of pages to standard output).
 - c) **addPages** - a function whose argument is a pointer to a structure of type **Book**. The function should add 10 to the number of pages in the passed argument.

Create a test case for each of the functions.

9. Write a structure **Laptop** with fields **model** (character array of length 30) and **price** (of type **float**). Then write the functions and call each of them at least once:
- a) **initLaptop** - a function that takes two arguments: **model** and **price**, and returns a newly-created structure setting its members from the passed arguments.
 - b) **showLaptop** - a function whose argument is a variable of type **Laptop**. The function should print a description of the passed argument (print the **model** and **price** to standard output).
 - c) **reducePrice** - a function whose argument is a pointer to a structure of type **Laptop**. The function should reduce the **price** by 5% in the passed argument.

Create a test case for each of the functions.

10. Create a structure **Airplane** with two fields **model** (string) and **number_of_engines** (any integer type). Then create a function whose arguments are a non-empty array of **Airplane** structures and the size of the array. The function should return the smallest number of engines. Create a test case.

11. Create a structure **Computer** with two fields **brand** (string) and **number_of_cores** (any integer type). Then create a function whose arguments are a non-empty array of **Computer** structures and the size of the array. The function should return the largest number of cores. Create a test case.
12. Create a structure **Book** with two fields: **title** (string) and **page_count** (int). Then create a function whose arguments are a non-empty array of **Book** structures and the size of the array. The function should return the book (as a structure) with the largest number of pages. In case of several books in the array with the largest number of pages, return the last possible one. Create a test case.
13. Write a structure **Car** with fields **brand** (character array of length 50) and **mileage** (of type int). Then write two functions and call each of them at least once:
 - a) **initCar** - a function that takes two arguments: brand and mileage, and returns a pointer to a newly created structure, setting its members from the passed arguments. Additionally, the function should check that the brand is a string of length at least 2 and the mileage is greater than 0. If at least one of the conditions is not satisfied, the function should return NULL.
 - b) **increaseMileage** - a function whose argument is a pointer to a structure of type **Car**. The function should add 1000 to the mileage in the passed argument.

Make sure you can call the second function in **main**.

14. Write a structure **Bike** with fields **model** (character array of length 40) and **distance_travelled** (of type int). Then write two functions and call each of them at least once:
 - a) **initBike** - a function that takes two arguments: bike model and distance travelled, and returns a pointer to a newly created structure, setting its values according to the passed arguments. The function should additionally check whether the model is a string of length at least 3 and the distance travelled is greater than 0. If at least one of the conditions is not satisfied, the function should return NULL.
 - b) **increaseDistance** - a function whose argument is a pointer to a structure of type **Bike**. The function should add 500 to the distance travelled.

Make sure you can call the second function in **main**.

15. Write a structure **Computer** with fields **manufacturer** (character array of length 30) and **usage_hours** (of type int). Then write two functions and call each of them at least once:
 - a) **initComputer** - a function that takes two arguments: manufacturer and number of usage hours, and returns a pointer to a newly created structure, setting its values according to the passed arguments. The function should also check whether the manufacturer's name has at least 2 characters and the number of hours is greater than 0. If both conditions are not jointly satisfied, the function should return NULL.

- b) **increaseUsage** - a function whose argument is a pointer to a structure of type **Computer**. The function should add 100 to the number of usage hours.

Make sure you can call the second function in **main**.

16. Write a structure **Apartment** with fields **address** (character array of length 100) and **number_of_residents** (of type **int**). Then write two functions and call each of them at least once:
 - a) **initApartment** - a function that takes two arguments: address and number of residents, and returns a pointer to a newly created structure, setting its values according to the passed arguments. The function should check whether the address has at least 5 characters and the number of residents is greater than 0. If both conditions are not jointly satisfied, the function should return **NULL**.
 - b) **increaseResidents** - a function whose argument is a pointer to a structure of type **Apartment**. The function should add 2 to the number of residents.

Make sure you can call the second function in **main**.

17. Create an enumeration type **Month** representing the months. Write a function **days_in_month()** that takes as an argument a value of type **Month** and returns the number of days in the given month. Ignore the leap year problem in this exercise. Create a test case for the function.
18. Create an enumeration type **Day** representing the days of the week. Write a function **print_days()** that takes as arguments a value of type **Day** and a number **n**. The function should print the current day of the week, and then recursively call itself with the next day of the week, decrementing **n** with each call, until **n** drops to 0. Create a test case for the function.
19. Create a union **Number** that allows simultaneous storage of an integer and a rational number. Then “wrap” the union in a structure and create a function that allows initializing the union’s fields from standard input. Create a test case for the function.
20. Create a structure **element** with two fields: **i** of type **int** and **next** being a pointer to the defined structure. In **main**, create several structural variables.

14 Singly-linked lists

Use the convention discussed in the lecture in the exercises.

14.1 Without functions

1. In `main`, create a headless list with three elements 4,5,-12. Then, on consecutive lines, display the elements of the list. Use the structure:

```
struct element {  
    int x;  
    struct element * next;  
};
```

2. In `main`, create a list with a head containing three elements 4,5,-12. Then, on consecutive lines, display the elements of the list. Use the structure:

```
struct element {  
    int x;  
    struct element * next;  
};
```

14.2 Displaying

3. Write a function `printListWithoutHead` that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should display the consecutive elements of the list on consecutive lines or display a message that the list is empty. Create a test case.

4. Write a function `printListWithHead` that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should display the consecutive elements of the list on consecutive lines or display a message that the list is empty. Create a test case.

5. Write a function `printPos` that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should display those elements of the list that are positive. Create a test case.

6. Write a function `printOdd` that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should display those elements of the list that are odd. Create a test case.

14.3 Creating a list

7. Write a no-argument function `createNoHead` that returns a pointer to a “newly-created” empty headless list. Create a test case. Use the structure:

```
struct element {  
    int x;  
    struct element * next;  
};
```

8. Write a no-argument function `createWithHead` that returns a pointer to a “newly-created” empty list with a head. Create a test case. Use the structure:

```
struct element {  
    int x;  
    struct element * next;  
};
```

14.4 Adding at the beginning

9. Write a function `addFirst` that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

and an integer `a`. The function should add a new element with value `a` at the beginning of the list. Create a test case.

10. Write a function `addFirst` that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

and an integer `a`. The function should add a new element with value `a` at the beginning of the list. Create a test case.

14.5 Adding at the end

11. Write a function `addLast` that takes as an argument a headless list with elements of type:


```
struct element {
    int x;
    struct element * next;
};
```

and an integer **a**. The function should add a new element with value **a** at the end of the list. Create a test case.

12. Write a function **addLast** that takes as an argument a list with a head with elements of type:

```
struct element {
    int x;
    struct element * next;
};
```

and an integer **a**. The function should add a new element with value **a** at the end of the list. Create a test case.

14.6 Searching/counters

13. Write a function **find** that takes as an argument a headless list with elements of type:

```
struct element {
    int x;
    struct element * next;
};
```

and an integer **a**. The function should return 1 if at least one element of the list is equal to **a**, and return 0 otherwise. Create a test case.

14. Write a function **find** that takes as an argument a list with a head with elements of type:

```
struct element {
    int x;
    struct element * next;
};
```

and an integer **a**. The function should return 1 if at least one element of the list is equal to **a**, and return 0 otherwise. Create a test case.

15. Write a function that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should return the sum of odd elements in the list. Create a test case.

16. Write a function that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should return the sum of negative elements in the list. Create a test case.

17. Write a function that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should return the address of the last even element or NULL if there is no such element or the list is empty. Create a test case.

18. Write a function that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should return the address of the last positive element or NULL if there is no such element or the list is empty. Create a test case.

14.7 Removing individual elements

19. Write a function that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the first element of the list (provided the list is not empty). Create a test case.

20. Write a function that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the first element of the list (provided the list is not empty). Create a test case.

21. Write a function that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the last element of the list (provided the list is not empty). Create a test case.

22. Write a function that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the last element of the list (provided the list is not empty). Create a test case.

23. Write a function that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the second-to-last element of the list (provided the list has at least two elements). Create a test case.

24. Write a function that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the second-to-last element of the list (provided the list has at least two elements). Create a test case.

14.8 Removing the entire list

25. Write a function that takes as an argument a headless list with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the entire list (manually, element by element). After removal, make sure the list is empty according to the convention. Create a test case.

26. Write a function that takes as an argument a list with a head with elements of type:

```
struct element {  
    int x;  
    struct element * next;  
};
```

The function should remove the entire list (manually, element by element). After removal, make sure the list is empty according to the convention. Create a test case.

14.9 Miscellaneous exercises

27. Write a function that takes as an argument a headless list with elements of type:

```
struct node {  
    int i;  
    struct node * next;  
};
```

The function should display on the console, on consecutive lines, the values of those elements in the list that are perfect squares of integers. Create a test case.

Example. If the list consists of the elements 4,5,6,-34,0,25,11, then 4,0,25 should be displayed on consecutive lines.

28. Write a function that takes as an argument a headless list with elements of type:

```
struct node {  
    double x;  
    struct node * next;  
};
```

and returns the smallest of the numbers in the list. In the case of an empty list, the function should return zero. Create one test case.

29. Write a function that takes as an argument a headless list with elements of type:

```
struct node {  
    int i;  
    struct node * next;  
};
```

The function should duplicate the value of the last element, provided the list is not empty. In the case of an empty list, the function should do nothing. Create a test case.

Example: For the list 3,4,5, the 5 should be duplicated, so after modification the list should have the form 3,4,5,5.

30. Write a function that takes as an argument a list with a head with elements of type:

```
struct elem {
    int a;
    struct elem * next;
};
```

The function should duplicate the value of the last element, provided the list is not empty. In the case of an empty list, the function should do nothing. Create a test case.

Example: For the list 3,4,5, the 5 should be duplicated, so after modification the list should have the form 3,4,5,5.

31. Write a function that compares two lists with a head with elements of type:

```
struct node {
    int i;
    struct node * next;
};
```

and returns 1 if the last elements of the lists are equal, and 0 otherwise (including when either of the lists or both are empty). Create one test case.

32. Write a function that compares two headless lists with elements of type:

```
struct node {
    int a;
    struct node * next;
};
```

and returns 1 if the lists are of the same length, and 0 otherwise (including when either of the lists or both are empty). Create a test case.

33. Write a function that takes two lists with a head with elements of type:

```
struct node {
    int value;
    struct node * next;
};
```

and returns 1 if the sum of all odd elements in both lists is the same, and 0 otherwise (including when either of the lists or both are empty). Create a test case.

34. Write a function that takes two headless lists with elements of type:

```
struct node {
    int w;
    struct node * next;
};
```

and returns 1 if the sum of all even elements in both lists is the same, and 0 otherwise (including when either of the lists or both are empty). Create a test case.

35. Write a function that takes as an argument a list with a head with elements of type:

```
struct node {
    int x;
    struct node * next;
};
```

The function should return the sum of odd elements from the list. Create one test case.

36. Write a function that takes as an argument a headless list with elements of type:

```
struct elem {
    int x;
    struct elem * next;
};
```

The function should double all positive elements in the list (if any exist). Create one test case.

37. Write a function that takes as an argument a list with a head with elements of type:

```
struct node {
    int y;
    struct node * next;
};
```

and two integers **a** and **b**. The function should add two new elements at the beginning of the list and set their values respectively from the given arguments. Create one test case.

38. Write a function that takes as an argument a headless list with elements of type:

```
struct node {
    int a;
    struct node * next;
};
```

and two integers **a** and **b**. The function should add two new elements at the beginning of the list and set their values respectively from the given arguments. Create one test case.

39. Write a function that takes as an argument a headless list with elements of type:

```
struct element {
    float value;
    struct element * next;
};
```

The function should replace each element's value with its absolute value. For an empty list, the function should do nothing. Create a test case.

40. Write a function that takes as an argument a list with a head with elements of type:

```
struct node {
    int val;
    struct node * next;
};
```

The function should replace each element's value with its absolute value. For an empty list, the function should do nothing. Create a test case.

41. Write a function that takes as an argument a list with a head with elements of type:

```
struct node {
    int x;
    struct node * next;
};
```

and an integer **d**. The function should return how many elements in the list are equal to **d**. Create one test case.

Bibliography and Other Problem Sets

Krzaczkowski, Jacek. 2011. *Zadania z Programowania w Języku c/c++, Cz. I*. Instytut Informatyki UMCS Lublin.